UNIVERSITÀ DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Elaborato finale in **Controlli Automatici**

REINFORCEMENT LEARNING PER IL CONTROLLO AUTONOMO DELL'ATTERRAGGIO DI UN VELIVOLO

Anno Accademico 2024/2025

Candidato
Davide Romano
matr. N46006993

Ai miei genitori, che mi permettono di inseguire i miei sogni,

professionali e non.

A mio fratello e mia sorella, che sono stati i miei primi amici.

Alla SSC NAPOLI, che in questi tre anni mi ha regalato 2 scudetti.

Indice

Introduzione **3**

1 Reinforcement learning **4**

1.1 Definizione e contesto 4

1.1.1 Rapporto agente-ambiente 6

1.1.2 Struttura dell'agente 7

1.2 Fasi del reinforcement learning 9

1.2.1 Modellazione dell'ambiente 9

1.2.2 Progettazione della ricompensa 9

1.2.3 Definizione della policy 12

1.2.4 Addestramento della policy 14

1.2.5 Deploy 15

2 Tipologie di algoritmi di einforcement learning **16**

2.1 Panoramic generale 16

2.2 Algoritmi Value-Based 18

2.2.1 Problema dello spazio di azioni continuo 18

2.3 Algoritmi Policy-Based 19

2.3.1 Metodi policy-gradient 20

2.3.2 Teorema del gradiente di policy 21

2.3.3 Problema del gradiente rumoroso 22
2.4 Algoritmi Actor-Critic 22
2.4.1 Errore di differenza temporale 23
2.4.2 Vantaggi 24

3 Proximal Policy Optimization 26
3.1 Origine 27
3.2 Idea chiave 28
3.2.1 Concetti introduttivi 29
3.2.2 Funzione obiettivo surrogata e clipping 32
3.3 Pseudocodice 34
3.3.1 Generalized Advantage Estimation (GAE) . . . 37

4 Caso di studio: atterraggio di un velivolo 39
4.1 Definizione dell'ambiente e formulazione della ricompensa 40
4.1.1 Ambiente come classe 41
4.1.2 Spazio degli stati 42
4.1.3 Vettore di osservazioni 43
4.1.4 Spazio delle azioni 44
4.1.5 Definizione funzione ricompensa 45
4.2 Creazione e addestramento dell'agente PPO 48
4.2.1 Architettura dell'agente 48
4.2.2 Scelta degli iperparametri 49
4.2.3 Istanziazione dell'agente 51
4.2.4 Procedura di addestramento 52
4.3 Simulazione e analisi 55
4.3.1 Procedura di simulazione 55

4.3.2 Analisi dei risultati 57

5 Conclusioni **59**

Bibliografia **64**

Introduzione

Da sempre il funzionamento della mente umana rappresenta un enigma affascinante: l'essere umano ha cercato di comprenderne i meccanismi, senza mai riuscire a svelarne del tutto i misteri, nemmeno oggi, nonostante gli straordinari progressi della scienza e della tecnologia. Nonostante questa conoscenza ancora incompleta, l'umanità ha scelto di spingersi oltre: porsi l'obiettivo di riprodurre in forma artificiale la propria intelligenza, o comunque qualcosa di simile.

Oggi l'intelligenza artificiale occupa un ruolo centrale nello sviluppo tecnologico e scientifico, e, soprattutto, sta diventando uno strumento di utilizzo quotidiano accessibile anche a chi non è esperto del settore. In poco tempo, l'intelligenza artificiale è passata dall'essere un'idea sperimentale a diventare una realtà tangibile, capace di trasformare radicalmente il modo in cui lavoriamo, comunichiamo ed affrontiamo problemi complessi.

All'interno del vasto panorama dell'intelligenza artificiale, il machine learning ha assunto un ruolo chiave, consentendo alle macchine di apprendere direttamente dai dati. Tra le diverse tecniche di ap-

prendimento, il reinforcement learning rappresenta una delle aree più promettenti. Esso si fonda sull'interazione continua con l'ambiente e ha l'obiettivo di sviluppare strategie di comportamento ottimali. Questa caratteristica lo rende particolarmente adatto ad affrontare problemi di controllo complessi, nei quali la macchina, proprio come un bambino che scopre e sperimenta il mondo, impara passo dopo passo: esplora, commette errori, riceve segnali di ricompensa e, gradualmente, sviluppa strategie sempre più efficaci. In questo senso, il reinforcement learning riproduce in modo artificiale un processo di apprendimento naturale, basato sull'esperienza diretta e sulla capacità di adattarsi a contesti in continuo cambiamento.

Questa tesi si colloca in tale contesto, proponendo un'analisi relativa all'algoritmo Proximal Policy Optimization (PPO), uno degli algoritmi più diffusi e performanti nel campo del reinforcement learning. L'obiettivo principale è stato quello di studiarne il funzionamento, verificarne i vantaggi rispetto ad approcci alternativi e valutarne l'efficacia in scenari specifici, sia teorici sia pratici.

Il lavoro si articola in vari capitoli. Nel Capitolo 1 verrà fornita una panoramica introduttiva dei concetti fondamentali del reinforcement learning. Nel Capitolo 2 si analizzeranno le principali tipologie di algoritmi di reinforcement learning, con particolare attenzione ai metodi di policy gradient e agli approcci actor-critic, evidenziandone i limiti che hanno portato allo sviluppo di tecniche più avanzate, come il PPO.

Il Capitolo 3 sarà dedicato all'approfondimento dell'algoritmo PPO, illustrandone i principi e un'idea del funzionamento. Infine, nel Capitolo 4 verrà presentato un esempio di implementazione dell'algoritmo in ambiente MATLAB, descrivendo il setup utilizzato, i parametri di addestramento, i risultati ottenuti e una breve analisi che evidenzia punti di forza e criticità.

Lo scopo del lavoro non è soltanto validare l'efficacia di PPO, ma anche offrire un contributo di comprensione e sintesi per chiunque sia interessato a esplorare il potenziale del reinforcement learning.

Chapter 1

Reinforcement learning

In questo capitolo verranno introdotti i concetti fondamentali del reinforcement learning, con l'obiettivo di fornire le basi per compprendere i capitoli successivi. Si introdurranno i concetti chiave: agente, ambiente, stato, azione, ricompensa e return e le funzioni di valore V e Q. Verrà inoltre spiegata la nozione di policy (deterministica e stocastica) e come viene rappresentata in pratica. Infine il capitolo illustra il flusso operativo tipico per risolvere un problema con il reinforcement learning: progettazione dell'ambiente, progettazione della ricompensa, scelta della policy, addestramento e deploy.

1.1 Definizione e contesto

Per prima cosa è utile fornire una definizione del reinforcement learning. Come riportato da Sutton e Barto in *Reinforcement Learning:*

An Introduction:

> "Il reinforcement learning consiste nell'apprendere cosa fare, ovvero come mappare le situazioni in azioni, in modo da massimizzare un segnale di ricompensa numerica. A chi apprende non viene detto quali azioni intraprendere, ma deve invece scoprire quali azioni producono la ricompensa maggiore provandole."[16]

Il reinforcement learning è una delle tre grandi categorie del **machine learning**, le quali verranno introdotte brevemente di seguito:

- **unsupervised learning**: vengono utilizzati algoritmi per analizzare e raggruppare set di dati non etichettati, scoprendo pattern o cluster nascosti senza la necessità di intervento umano

- **supervised learning**: vengono impiegati set di dati di input–output etichettati per addestrare modelli che apprendono la relazione sottostante e possono quindi prevedere l'output corretto su nuovi dati non etichettati.

- **reinforcement learning**: A differenza degli altri due framework di apprendimento, che operano utilizzando un set di dati statico, il reinforcement learning lavora con dati provenienti da un **ambiente dinamico**. Qui l'obiettivo non è raggruppare i dati o etichettarli, ma trovare la migliore sequenza di azioni che

generi il risultato ottimale. Il modo in cui il reinforcement learning risolve questo problema è consentire a un software, chiamato agente, di esplorare, interagire e apprendere dall'ambiente

Inoltre il reinforcement learning può essere applicato a **problemi complessi di controllo**, come l'atterraggio di un velivolo. A differenza dell'approccio classico, che suddivide il problema in moduli distinti collegati tra loro, il reinforcement learning consente di concentrare il controllo in un'unica funzione, ovvero l'agente, il quale riceve le osservazioni dall'ambiente e produce direttamente le azioni. Va però sottolineato che questa soluzione non è sempre più efficiente rispetto ai metodi tradizionali [6].

1.1.1 Rapporto agente-ambiente

Il reinforcement learning si basa sull'interazione ciclica tra agente e ambiente che può essere anche infinita. Ad ogni interazione di questo ciclo l'agente decide un'**azione** da compiere, che avrà un effetto sull'ambiente, il quale passerà ad un nuovo **stato** e produrrà una **ricompensa**. Stato e ricompensa verranno forniti all'agente per migliorare le decisioni future.

Nell'interagire con l'ambiente l'agente deve bilanciare **esplorazione** e **sfruttamento**. L'esplorazione consiste nel provare nuove azioni per raccogliere informazioni sull'ambiente, mentre lo sfruttamento consiste nello scegliere l'azione migliore sfruttando solo le informazioni attuali

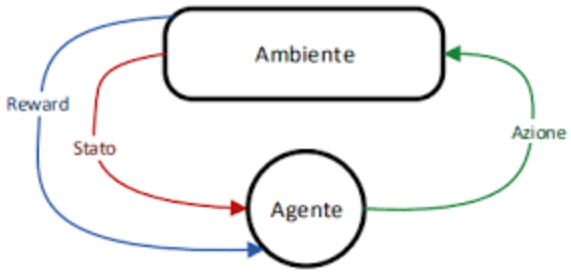


Figure 1.1: Ciclo d'interazione agente-ambiente [4]

sull'ambiente. Un buon bilanciamento è fondamentale: troppa esplorazione può rallentare l'apprendimento, troppo sfruttamento può far perdere le soluzioni migliori [6].

1.1.2 Struttura dell'agente

Nel reinforcement learning l'**agente** è il componenete attivo che interagisce con l'ambiente, scegliendo azioni e ricevendo osservazioni e ricompense, con l'obiettivo di massimizzare la ricompensa complessiva che può ottenere.

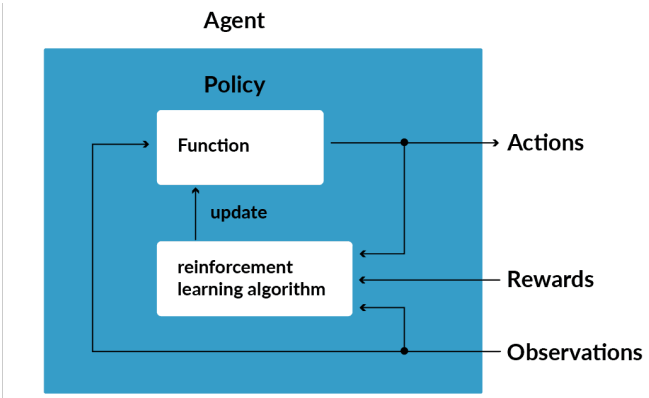


Figure 1.2: Struttura interna di un agente generico di reinforcement learning, con indicazione dei segnali in ingresso nelle varie componenti e dei risultati in uscita verso l'ambiente[1]

La struttura interna dell'agente è composta da due elementi fondamentali:

- **Policy (π)**: La policy è la funzione che mappa le osservazioni sullo stato corrente dell'ambiente in azioni, essa quindi è la **strategia di comportamento** dell'agente. La policy può essere una semplice tabella, che associa ad ogni stato s un'azione, ma tale approccio non può essere utilizzato nel caso in cui abbiamo a che fare con spazio deglli stati e/o spazio delle azioni continui. Anche utilizzare una semplice funzione come policy può risultare complesso, poiché è difficile definire in anticipo una struttura adeguata per rappresentarla correttamente. Per questi motivi, in genere, si utilizza come policy una **rete neurale** [1] che funge da approssimatore generale di funzione [6].

- **Algoritmo di apprendimento**: L'algoritmo di apprendimento è ciò che consente all'agente di migliorare la sua policy nel tempo. Questo componente modifica la policy in base alle azioni intraprese, alle osservazioni e all'importo della ricompensa riscossa [6].

Quindi l'obiettivo dell'agente è quello di utilizzare algoritmi di reinforcement learning per apprendere la policy migliore mentre interagisce con l'ambiente in modo che, dato qualsiasi stato, intraprenda sempre l'azione più ottimale.

[1] Una rete neurale artificiale è un modello computazionale formato da nodi interconnessi, detti neuroni artificiali, che elaborano informazioni in modo cooperativo per apprendere e risolvere compiti complessi

1.2 Fasi del reinforcement learning

E' importante andare a descrivere per bene il flusso di lavoro associato alla risoluzione di un problema complesso sfruttando il reinforcement learning. In generale, ciò richiede l'intervento su cinque aree diverse [6], che verranno presentate in questo paragrafo.

1.2.1 Modellazione dell'ambiente

L'ambiente è tutto ciò che esiste al di fuori dell'agente, è dove l'agente invia le azioni ed è ciò che genera ricompense e osservazioni che permettono all'agente di apprendere. In questa fase bisogna scegliere cosa includere nell'ambiente e cosa escludere da esso.

Bisogna inoltre scegliere se si tratta di una **simulazione** o di una **configurazione fisica**. Gli ambienti simulati sono il metodo più comune per addestrare un agente, perchè permettono di velocizzare il processo di apprendimento e facilitano la modellazione di situazioni che sarebbero difficili da testare, come ad esempio scenari che potrebbero causare danni all'hardware. Al contempo gli ambienti reali garantiscono un apprendimento più preciso e affidabile [6]

1.2.2 Progettazione della ricompensa

"Il segnale di ricompensa definisce l'obiettivo del problema specifico di reinforcement learning" [16]. In altri termini, la ricompensa è una

funzione che restituisce un valore numerico in grado di misurare quanto sia vantaggioso per l'agente trovarsi in un certo stato ed eseguire una determinata azione rispetto all'obiettivo prefissato. È proprio questo segnale che guida l'agente durante l'apprendimento.

Nel reinforcement learning l'obiettivo dell'agente è massimizzare la quantità totale di ricompense ottenute nel tempo, chiamata **return**. Il return può essere definito in modi diversi a seconda che l'interazione tra agente e ambiente sia modellata come un episodio finito o infinito. Un **episodio** rappresenta la sequenza completa di interazioni agente-ambiente, che inizia da uno stato iniziale e termina quando si verifica una condizione di fine [16]. Quindi il return può essere definito come:

- **Finite-horizon undiscounted return**: negli **episodi finiti**, l'interazione tra agente e ambiente termina dopo un certo numero di passi T. In questo caso, il return corrisponde alla somma delle ricompense accumulate durante l'intero episodio:

$$R(\tau) = \sum_{t=0}^{T} r_t \tag{1.1}$$

  τ è definita **traiettroia**: sequenza finita di stati e azioni che descrive l'interazione agente-ambiente dall'inizio alla fine di un episodio, mentre t indica un **passo** dell'episodio, singola interazione agente-ambiente che fornisce la ricompensa r_t [17].

- **Infinite-horizon discounted return**: in scenari di **episodio**

infinito, l'interazione tra agente e ambiente non ha una condizione di termine fissata e può proseguire indefinitamente. Per rendere gestibile questo caso, si introduce il **fattore di sconto** $\gamma \in (0, 1)$:

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t \tag{1.2}$$

L'uso di γ ha un duplice motivo: viene garantito che la serie converga e permette di dare un maggior peso alle ricompense a breve termine, rispetto alle stesse ma lontane nel tempo [17].

Il return non è direttamente utilizzabile durante l'addestramento poichè è disponibile solo dopo aver completato un episodio interamente. Per questo motivo si introducono le **funzioni di valore** che forniscono una stima del return atteso [6]. Queste sono di due tipi:

- **Valore di uno stato**: valore atteso del return totale partendo dallo stato s, seguendo la policy π:

$$V^{\pi}(s) = E_{\pi}\left[R(\tau) \mid s_0 = s\right][17] \tag{1.3}$$

- **Valore di stato-azione**: valore atteso del return totale partendo dallo stato s, eseguendo subito l'azione a, e seguendo poi la policy π:

$$Q^{\pi}(s, a) = E_{\pi}\left[R(\tau) \mid s_0 = s,\, a_0 = a\right][17] \tag{1.4}$$

Il return e il valore dipendono direttamente dalla funzione di ricompensa, perciò quest'ultima è fondamentale. Essa deve essere formulata a seconda del contesto. Crearla è semplice, in quanto non ci sono vincoli sul tipo di funzione, ma crearne una buona è molto complesso. Infatti, se la funzione ricompensa è mal progettata, l'agente potrebbe imparare comportamenti indesiderati o subottimli. Inoltre deve essere progettata in modo da bilanciare le prestazioni ottenute rispetto allo sforzo o costo richiesto per ottenerle [6].

L'algoritmo di apprendimento non deve basarsi solo sulla ricompensa istantanea, ma anche sulla stima del valore degli stati futuri, bilanciando ricompense immediate e a lungo termine. Essere troppo "miopi", cioè considerare soltanto il beneficio immediato, può portare a soluzioni subottimali; al contrario, dare troppo peso al lungo termine può introdurre incertezza, poiché le stime del valore diventano sempre meno affidabili più ci si allontana dall'istante attuale [6].

1.2.3 Definizione della policy

È stato già discusso in precedenza cosa sia la policy e possiamo riassumere dicendo che essa è una funzione che indica quale azione eseguire nello stato s_t. Si possono riconoscere due tipologie di policy:

- **Deterministiche**: Sono funzioni del tipo:

$$\pi(s) = a$$

Con questo approccio non c'è casualità: la policy associa ad ogni stato un'unica azione ben precisa [16].

- **Stocastiche**: Sono funzioni del tipo:

$$\pi(a \mid s) = P[a_t = a \mid s_t = s]$$

Con questo approccio l'azione viene scelta in base ad una **distribuzione di probabilità** [2], ovvero, per ogni stato s, la policy assegna a ciascuna azione a una probabilità e l'agente seleziona le azioni campionando da questa distribuzione. Nel seguito, per semplicità, verrà indicata la policy stocastica anche come $\pi(a, s)$ al posto di $\pi(a \mid s)$ [16].

Una policy deterministica è più semplice e diretta, ma una policy stocastica è più flessibile e favorisce l'esplorazione, il che la rende particolarmente utile specialmente in fase di addestramento [6].

Com già anticipato, negli algoritmi moderni di reinforcement learning si utlizza una rete neurale come policy, soprattutto quando si ha a che fare con spazi di stato o di azione grandi o continui. Poichè la rete si allena e impara da sola sembrerebbe che questa fase non richiede interventi; in realtà l'efficacia di una rete dipende fortemente da scelte strutturali fatte a monte [6].

Scelte importanti da prendere sono:

[2]Una distribuzione di probabilità è una funzione o legge che descrive quanto probabilmente ciascun possibile valore di una variabile si verifica

- **dimensione dello strato nascosto**: strato di neuroni posto tra l'input e l'output che trasforma e combina i dati in ingresso attraverso pesi e funzioni di attivazione. Tale scelta influisce sulla capacità di modellare funzioni complesse.

- **Funzioni di attivazione**: come ReLU [3] o sigmoid[4], che permettono di introdurre non-linearità nelle reti neurali, aumentando l'espressività del modello.

- **Struttura geneale della rete**: ad esempio, una rete completamente connessa oppure una rete con architetture specializzate.

Queste scelte, se non prese correttamente, possono portare ad avere una rete troppo semplice, la quale avrà difficoltà ad apprendere un comportamento ottimale, o troppo complessa, la quale rischia di essere difficile da addestrare.

1.2.4 Addestramento della policy

In questa fase l'obiettivo è apprendere la policy ottimale π^*, ovvero la **strategia che massimizzi il return**. Per fare ciò bisogna agire sulla policy in modo tale da modificarla iterativamente sulla base dell'esperienza raccolta, in modo da avvicinare sempre più la policy attuale a quella ottimale.

[3] funzione di attivazione che restituisce 0 per input negativi e l'input stesso se positivo
[4] funzione di attivazione che trasforma un valore reale in un output compreso tra 0 e 1

Tuttavia, il come e il perchè avvengono le modifiche dipende fortemente dall'algoritmo di apprendimento adottato. Esistono alcuni algoritmi che si basano esclusivamente sulle funzioni di valore, altri che aggiornano direttamente la policy, e altri ancora che combinano entrambi gli approcci. Una panoramica dei principali algoritmi di apprendimento sarà presentata nel capitolo successivo.

1.2.5 Deploy

L'ultimo passaggio del flusso di lavoro del reinforcement learning è l'implementazione della policy, ovvero rendere operativa, nel contesto reale, la strategia che l'agente ha appreso durante la fase di training. Se l'apprendimento avviene tramite l'agente fisico nell'ambiente reale, la policy appresa è già presente sull'agente e può essere sfruttata. Purtroppo, in genere la maggior parte dell'addestramento viene svolta in ambienti simulati e la policy che viene appresa potrebbe non essere perfettamente applicabile alla realtà fisica. Di conseguenza, prima che la policy venga utilizzata con successo in un sistema fisico reale, è spesso necessario un processo di adattamento sul dispositivo reale. Questo processo serve a colmare il divario tra la simulazione e la realtà. L'agente continua ad apprendere anche nell'ambiente reale, raffinandosi grazie a nuove esperienze dirette [6].

Chapter 2

Tipologie di algoritmi di einforcement learning

In questo capitolo verranno presentate le principali tipologie di algoritmi di reinforcement learning, a partire da una panoramica generale che distingue tra approcci model-based e model-free, fino a esplorare più in dettaglio le famiglie di algoritmi value-based, policy-based e actor-critic. Questa analisi fornisce le basi teoriche necessarie per comprendere algoritmi più avanzati come il Proximal Policy Optimization (PPO), approfondito nei capitoli successivi.

2.1 Panoramic generale

Gli algoritmi di reinforcement learning si suddividono principalmente in due grandi categorie: **Model-based**, che fanno uso di un modello

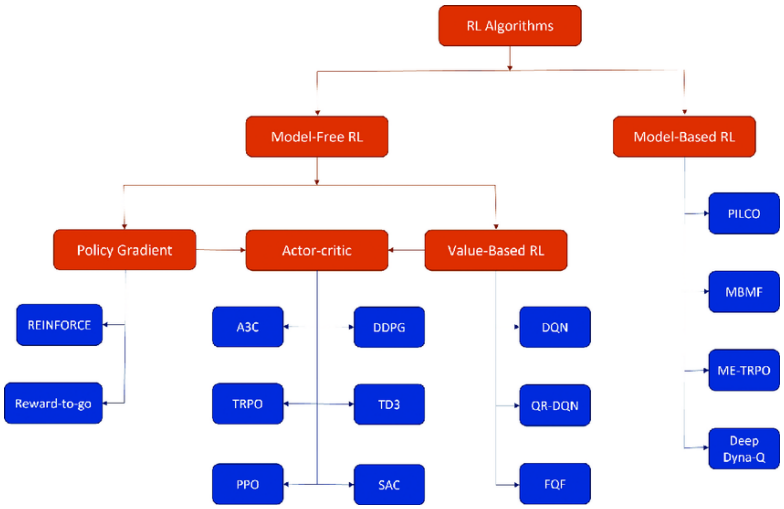


Figure 2.1: Una possibile tassonomia degli algoritmi di reinforcement learning [18]

dell'ambiente, e **Model-free**, che non fanno uso di alcun modello [10].

Gli algoritmi Model-based utilizzano un modello dell'ambiente, fornito o appreso, che permette di prevedere transizioni e ricompense, portando l'agente verso regioni ad alta ricompensa senza la necessità di esplorare tutto l'ambiente, ciò riduce il numero di interazioni agente-ambiente necessarie. Tuttavia, nella pratica si preferiscono spesso gli algoritmi Model-free, perché non richiedono un modello accurato dell'ambiente, sono più semplici da implementare e più adattabili a diversi contesti, pur necessitando di molte più interazioni per apprendere una policy efficace [13].

L'analisi si concentrerà sugli algoritmi model-free, in quanto, oltre ad essere i più diffusi, rappresentano la classe di appartenenza dell'algoritmo Proximal Policy Optimization.

2.2 Algoritmi Value-Based

Gli algoritmi value-based si concentrano sulla stima di una funzione di valore, questa può essere la funzione valore di uno stato $V(s)$ 1.3 o la funzione valore della coppia stato-azione $Q(s, a)$ 1.4. A partire da queste stime si costruisce una policy che sceglie le azioni che massimizzano tale stima, ad esempio considerando la funzione Q si ottiene:

$$\pi(s) = \arg\max_a Q(s, a) \tag{2.1}$$

Con questa policy si valuta quanto vale ogni possibile scelta e si sceglie l'azione con il valore stimato più alto [16].

Il principio che permette di stimare i valori della funzione di valore in modo coerente è l'**equazione di Bellman**, una relazione ricorsiva che mette in relazione valori a tempi diversi e permette di definire aggiornamenti iterativi per ottenere le stime corrette. Quindi i metodi value-based aggiornano iterativamente la funzione di valore sfruttando le varie transizioni osservate, spesso con qualche meccanismo di esplorazione per non rimanere bloccati in soluzioni sub-ottime [16].

2.2.1 Problema dello spazio di azioni continuo

Il procedimento appena descritto è particolarmente naturale quando lo spazio delle azioni è discreto, perché basta valutare $Q(s, a)$ su tutte le azioni e prendere l'arg max, tale operazione ha un costo di $O(K)$,

con K numero di azioni. Se invece lo spazio delle azioni è continuo non è possibile enumerare tutte le azioni e diventa complesso trovare l'arg max. Per questo motivo, in presenza di azioni continue, si preferiscono approcci che apprendono direttamente una policy parametrica (policy-based o actor-critic), che permette di rappresentare infinite azioni con pochi parametri [6].

2.3 Algoritmi Policy-Based

Gli algoritmi Policy-Based hanno l'obiettivo di modellare e ottimizzare direttamente la policy dell'agente, tipicamente parametrizzata attraverso i **parametri** θ. I parametri θ sono i pesi e bias della rete neurale che rappresenta la policy. Quindi modificando θ, si cambia la rete neurale e di conseguenza il comportamento dell'agente: nelle policy deterministiche vengono modificate le azioni fisse associate agli stati, mentre nelle policy stocastiche cambia la distribuzione di probabilità con cui le azioni vengono scelte. Nei metodi Policy-Based si agisce direttamente sui parametri θ, un insieme finito di valori, per migliorare la performance dell'agente, dato che la ricompensa dipende dalla policy stessa [6].

2.3.1 Metodi policy-gradient

I metodi più utilizzati in questo ambito sono quelli basati sul gradiente di policy.

La formula che viene utilizzata in questi metodi per l'aggiornamento dei parametri è la seguente:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)[16] \tag{2.2}$$

dove:

- $J(\theta)$ **è la funzione obiettivo**: misura quanto è buona la policy con i parametri θ attuali. Di solito essa coincide con il return atteso, ovvero la media del return su tutte le possibili traiettorie generate dalla policy corrente:

$$J(\theta) = E_{\tau \sim \pi_\theta}\big[R(\tau)\big] \tag{2.3}$$

  dove $R(\tau)$ è il return definito in 1.1 o 1.2.

- $\nabla_\theta J(\theta)$ **è il gradiente della funzione obiettivo**: indica come varia la funzione obiettivo $J(\theta)$ al variare dei parametri θ. In altre parole, fornisce la direzione nello spazio dei parametri lungo la quale $J(\theta)$ cresce più velocemente.

- α **è il learning rate**: un parametro compreso tra 0 e 1 che controlla l'ampiezza dei passi di aggiornamento. Valori troppo

grandi possono rendere l'apprendimento instabile, mentre valori troppo piccoli lo rendono molto lento [16].

Poiché stiamo usando una funzione di score e non una di costo, per migliorare i parametri θ si utilizza la **salita del gradiente** e non la discesa: stiamo sommando e non sottraendo [16]. Il gradiente infatti indica la direzione in cui bisogna modificare θ per aumentare $J(\theta)$.

Gli algoritmi che sfruttano la salita del gradiente alla fine di ogni ciclo di addestramento calcolano $J(\theta)$ e successivamente il gradiente $\nabla_\theta J(\theta)$, per poi aggiornare i parametri θ utilizzando la regola di aggiornamento 2.2.

2.3.2 Teorema del gradiente di policy

Per calcolare il gradiente si utilizza il **Teorema del gradiente di policy**. Senza discutere della complessa dimostrazione, il teorema afferma che per qualsiasi policy differenziabile [1] $\pi_\theta(s, a)$ il gradiente è pari a:

$$\nabla_\theta J(\theta) = E_{\pi_\theta}\big[\nabla_\theta \log \pi_\theta(s, a)\, Q^{\pi_\theta}(s, a)\big]\,[19] \tag{2.4}$$

Senza questa formula sarebbe quasi impossibile calcolare il gradiente $\nabla_\theta J(\theta)$, perchè per farlo dovremmo derivare anche rispetto alla distribuzione degli stati visitati dall'agente, dato che quest'ultima influenza la funzione obiettivo. Essa varia in funzione della policy e

[1] Una policy differenziabile è una policy $\pi_\theta(a|s)$ la cui funzione, rispetto ai parametri θ, è continua e derivabile.

quindi dipende anch'essa dai parametri θ, ma trovare la derivata della distribuzione degli stati è nella pratica impossibile [19].

2.3.3 Problema del gradiente rumoroso

Il problema degli algoritmi policy-based è che la policy viene aggiornata solo in base alle ricompense ottenute, senza fare stime intermedie del valore degli stati o delle azioni. Ogni singolo episodio può influenzare molto l'aggiornamento facendo sembrare buone azioni che in realtà non lo sono. Questo è chiamato **gradiente ruomoroso** e porta ad un apprendimento instabile e lento. In parte ciò viene migliorato scegliendo correttamente il learning rate α, ma è con gli algoritmi actor-critic che il problema viene ridotto in maniera più efficace, pur non essendo eliminato del tutto [16].

2.4 Algoritmi Actor-Critic

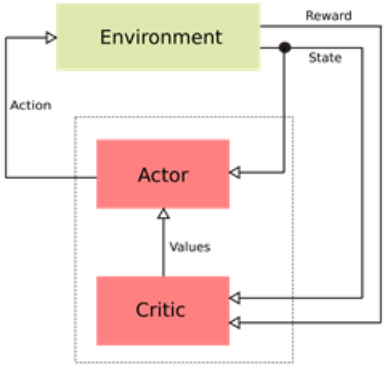


Figure 2.2: Funzionamento metodo actor-critic [3]

Gli algoritmi Actor-Critic combinano i due approcci appena visti [6]. In questo caso l'agente è formato da due reti neurali:

1. **Actor**: rappresenta la policy parametrizzata π_θ e si occupa di scegliere quale azione intraprendere. Essenzialmente il suo obbiettivo è di imparare una policy ottima per far muovere l'agente. E' evidente come questa parte sia ispirata ai metodi policy-based.

2. **Critic**: stima una funzione di valore, come $V^\pi(s)$ o $Q^\pi(s, a)$, per determinare quanto buona è stata l'azione scelta per guidare l'actor. E' evidente come questa parte sia ispirata ai metodi value-based.

Inoltre il Critic ha il compito di calcolare l'**errore** tra la ricompensa attesa e quella effettivamente ottenuta, errore che servirà per l'aggiornamento delle due reti neurali.

2.4.1 Errore di differenza temporale

Il tipo di errore calcolato più comune per praticità è il **TD (Temporal Difference error)** :

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)[16] \tag{2.5}$$

dove:

- r_t è la **ricompensa corrente** ricevuta al passo t.

- $V(s_t)$ è il **valore stimato dello stato corrente** s_t, cioè quanto ci si aspetta che sia buona la situazione attuale.

- $V(s_{t+1})$ è il **valore stimato dello stato successivo** s_{t+1}.

- $\gamma \in [0, 1]$ è il **fattore di sconto**, che determina quanto peso dare alle ricompense future rispetto a quelle immediate, già visto in 1.2.

Quindi sia l'actor che il critic si aggiornano ad ogni passo in base al risultato del calcolo dell'errore, in particolare:

- **Actor:** viene aggiornata la policy π. Se l'azione è stata migliore del previsto ($\delta_t > 0$) si aumenta la probabilità di sceglierla di nuovo, se peggiore ($\delta_t < 0$) si riduce.

- **Critic:** viene aggiornata la stima del valore $V(s_t)$. Se $\delta_t > 0$, significa che la ricompensa effettiva è stata maggiore della previsione, quindi $V(s_t)$ viene aumentato; se $\delta_t < 0$, la ricompensa è stata minore della previsione e $V(s_t)$ viene diminuito.

2.4.2 Vantaggi

Gli algoritmi Actor-Critic rappresentano la base per molti algoritmi moderni di reinforcement learning, come il PPO che analizzeremo nel prossimo capitolo. Questo è dovuto ai numerosi vantaggi che presentano:

- **Riduzione della varianza rispetto ai policy gradient puri**: l'introduzione del critico e quindi di una stima dei valori degli stati e delle azioni rende gli aggiornamenti della policy più stabili e mirati [16].

- **Gestione di spazi d'azione continui rispetto ai metodi value-based puri**: l'attore ha una policy parametrica che sceglie un'azione e il critic dovrà solo stabilire la qualità della singola azione, permettendo di aggiornare la policy senza dover considerare tutte le possibili azioni [6].

- **Flessibilità**: gli Actor-Critic possono combinare diverse architetture per attore e critico, adattandosi a vari tipi di problemi e ambienti.

- **Compatibilità con tecniche avanzate**: è possibile integrare tecniche avanzate, come l'advantage estimation (GAE) e il clipping, come avviene nel PPO, per rendere ancora più efficienti gli algoritmi.

Chapter 3

Proximal Policy Optimization

In questo capitolo verrà presentato il Proximal Policy Optimization (PPO), uno degli algoritmi più noti nel campo del reinforcement learning. L'obiettivo non è descriverlo in maniera esaustiva, vista la sua complessità, ma fornire una panoramica dei concetti fondamentali per comprenderne il funzionamento.

Verrà illustrata l'idea chiave alla base di PPO, ovvero come bilanciare la massimizzazione della ricompensa con la limitazione dei cambiamenti della policy. Inoltre si descriverà la funzione obiettivo surrogata clippata e il suo effetto sul gradiente e sulla stabilità dell'addestramento e infine si presenterà lo pseudocodice dell'algoritmo.

3.1 Origine

Il **Proximal Policy Optimization (PPO)** è stato introdotto nel 2017 da John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford e Oleg Klimov di OpenAI[1], nel paper "Proximal Policy Optimization Algorithms" [15].

Il PPO nasce per rendere più stabile e robusto l'apprendimento degli algoritmi basati sul gradiente di policy, i quali sono molto sensibili agli **iperparametri**, ovvero quei parametri che vengono impostati all'inizio per guidare l'apprendimento, come il learning rate. In teoria, una corretta impostazione dei valori di questi parametri potrebbe garantire stabilità e robustezza, ma nella pratica questa scelta è complessa e spesso poco generalizzabile. L'ottimizzazione diventa quindi fragile: piccole variazioni negli iperparametri possono portare a instabilità o a una convergenza verso la policy ottimale molto lenta. Per superare questo limite, si cerca un approccio alternativo che introduca una forma di vincolo sulla variazione della policy [15].

Per rispondere a questa esigenza è stato introdotto, sempre da ricercatori di OpenAI, **TRPO** (Trust Region Policy Optimization), che blocca variazioni eccessive della policy tra due aggiornamenti successivi, imponendo un vincolo che la mantenga all'interno di una trust region. Tuttavia, TRPO, pur essendo stabile, presenta una formulazione matematica complessa e costosa da implementare [15].

[1]OpenAI è un'azienda che si occupa di ricerca e sviluppo dell'intelligenza artificiale

Per superare anche questo limite è stato proposto **PPO**, un **algoritmo actor-critic** che ha sempre come obiettivo il controllare la variazione della policy, ma in modo più semplice ed efficiente. PPO sostituisce i vincoli complicati di TRPO con una funzione obiettivo surrogata "clippata", che penalizza automaticamente aggiornamenti troppo grandi [15]. In questo modo si ottiene un algoritmo più stabile dei classici policy gradient, ma anche più semplice e pratico da applicare rispetto a TRPO. Nel corso del capitolo diventerà chiaro cosa si intenda per funzione obiettivo "clippata".

3.2 Idea chiave

Come già dichiarato l'obiettivo principale di PPO è bilanciare due aspetti chiave:

- **Efficienza nell'apprendimento**: cercare di garantire che la policy migliori in modo consistente e significativo a ogni aggiornamento.
- **Limitare gli aggiornamenti**: vincolare cambiamenti troppo ampi della policy, che potrebbero destabilizzare l'apprendimento, mantenendo ogni aggiornamento entro un certo intervallo sicuro.

L'idea alla base di PPO è integrare questo vincolo direttamente all'interno della funzione obiettivo.

3.2.1 Concetti introduttivi

Quindi il passo fondamentale di questo algoritmo è impostare la funzione obiettivo. Prima di presentarla è utile introdurre alcuni concetti fondamentali:

- **Il rapporto di probabilità** $r_t(\theta)$

  Definito come:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}[10], \tag{3.1}$$

  misura quanto la nuova policy π_θ, in seguito all'aggiornamento dei parametri per salita del gradiente, cambia rispetto alla vecchia $\pi_{\theta_{\text{old}}}$ nel dare probabilità all'azione a_t nello stato s_t. Questo rapporto serve come fattore di ponderazione: se la nuova policy si discosta significativamente dalla vecchia, ovvero se $r_t(\theta)$ si allontana molto da 1, l'aggiornamento può essere moderato per evitare cambi troppo drastici.

- **L'Advantage** $\hat{A}_t$

  L'Advantage quantifica quanto un'azione è migliore o peggiore della media nello stato in cui è stata presa:

$$\hat{A}_t = Q(s_t, a_t) - V(s_t)[16] \tag{3.2}$$

dove $Q(s_t, a_t)$ è il valore atteso totale seguendo quell'azione e $V(s_t)$ è il valore medio dello stato.

- Se $\hat{A}_t > 0$, quell'azione è più vantaggiosa della media.
- Se $\hat{A}_t < 0$, è meno vantaggiosa.

L'Advantage serve a guidare la policy verso azioni migliori e allontanarla da quelle peggiori.

- **La media empirica $\hat{E}_t$**

  La media empirica è un metodo per stimare la media di una funzione usando solo i dati raccolti dall'ambiente e non tutte le possibili combinazioni. Quindi, invece di calcolare la media teorica su tutti gli stati e azioni possibili, si calcola la media sui campioni effettivamente osservati:

$$\hat{E}_t[f(s_t, a_t)] = \frac{1}{N} \sum_{t=1}^{N} f(s_t, a_t), \tag{3.3}$$

  dove N è il numero di transizioni osservate e $f(s_t, a_t)$ è la quantità di interesse. Questo permette di utilizzare in modo pratico le esperienze raccolte per stimare gli effetti degli aggiornamenti della policy.

Inoltre nel PPO si parla di **funzione obiettivo surrogata** perché viene utilizza una versione semplificata della vera funzione di performance della policy, vista in (2.3). Quest'ultima dipende dall'aspettativa

su tutte le possibili traiettorie, spesso impossibile da calcolare esattamente [15]. In altre parole, la funzione surrogata è un proxy pratico e calcolabile della funzione di performance vera.

Si può ora introdurre una prima versione della funzione obiettivo (utilizzata, ad esempio, dal TRPO):

$$L(\theta) = \widehat{E}_t\big[r_t(\theta)\,\widehat{A}_t\big]\,[15] \tag{3.4}$$

Questa combina gli elementi visti in precedenza e permette alla policy di apprendere in modo mirato, premiando le azioni vantaggiose. Il problema di questa funzione obiettivo è che se la probabilità di un'azione aumenta troppo rispetto alla vecchia policy ciò porterà ad un gradiente molto ripido, portando ad una policy molto diversa dall'attuale, potenzialmente rendendo instabile l'addestramento [15].

La funzione obiettivo che viene utilizzata dal PPO per rispondere a questo problema sfrutta allora due funzioni: **`min`**, la quale restituisce il minimo tra due valori al suo interno, e la funzione **`clip`**, che ha il seguente comportamento per valori scalari:

$$\text{clip}(x, a, b) = \min\{\max\{x, a\}, b\} = \begin{cases} a, & x < a, \\ x, & a \le x \le b, \\ b, & x > b. \end{cases} \tag{3.5}$$

Come si può notare questa funzione prende in input un numero x e

due soglie a (minimo) e b (massimo), restituendo x se si trova tra i due estremi, altrimenti lo "taglia" (clip) al limite più vicino. È quindi una proiezione dell'input sull'intervallo $[a, b]$.

3.2.2 Funzione obiettivo surrogata e clipping

Si è pronti finalmente a mostrare la funzione obiettivo surrogata utilizzata dal PPO:

$$L^{\text{CLIP}}(\theta) = \hat{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big] [15] \quad (3.6)$$

In pratica, il primo termine all'interno del minimo coincide con quello della funzione obiettivo (3.4) vista in precedenza. Il secondo termine, invece, introduce la correzione tramite il **clipping**, quindi vincola il rapporto $r_t(\theta)$ ad assumere solo valori all'interno dell'intervallo $[1 - \epsilon,\ 1 + \epsilon]$.

Per capire l'effetto del clipping sull'aggiornamento, possiamo distinguere due casi:

- $\mathbf{A > 0}$: l'azione è buona e $\pi_\theta(a_t, s_t)$ tende ad aumentare. Per evitare che la probabilità dell'azione nella nuova policy cresca troppo rispetto alla vecchia, il rapporto $r_t(\theta)$ viene tagliato ad $1 + \epsilon$. Questo perché un singolo update non dovrebbe provocare cambiamenti troppo elevati. In pratica, se l'azione era buona ed è diventata ancora più probabile seguendo il gradiente, è meglio

non continuare troppo ad aggiornarla in quella direzione [15].

- **A** < 0: l'azione è sfavorevole e $\pi_\theta(a_t, s_t)$ tende a diminuire, ma il clipping evita di ridurre $r_t(\theta)$ al di sotto di $1 - \epsilon$, prevenendo che un'azione già poco probabile diventi ancora meno probabile [15]. Come si può notare dalla figura 3.1, essendo A negativo, la moltiplicazione per questo valore ribalta il grafico rispetto al caso precedente [15].

Il parametro ϵ, detto **fattore di clip**, assume valori nell'intevallo $[0, 1]$ e controlla quanto la nuova policy può differire dalla precedente. Tipicamente si sceglie $\epsilon = 0.1$ oppure $\epsilon = 0.2$: valori più piccoli rendono l'apprendimento più stabile ma più lento, mentre valori più grandi accelerano l'aggiornamento a scapito della stabilità [15].

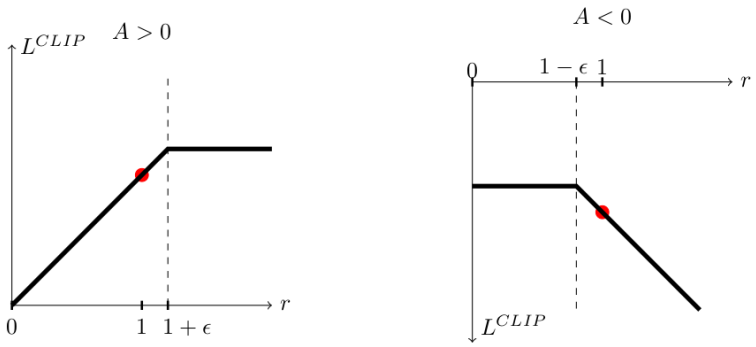


Figure 3.1: rappresentazione grafica di un singolo aggiornamento della funzione L^{CLIP} in funzione del rapporto di probabilità r, con l'Advantage positivo a sinistra e negativo a destra. Il puntino rosso segna l'inizio dell'ottimizzazione, corrispondente a $r = 1$, cioè quando la nuova policy coincide ancora con quella precedente[15]

In termini di gradiente, quando il rapporto $r_t(\theta)$ supera i limiti imposti dal clipping, la funzione obiettivo smette di crescere o diminuire

rispetto a θ. Poiché il valore della funzione non cambia più al variare di θ, il gradiente diventa zero, bloccando ulteriori aggiornamenti per quell'azione.

In sintesi, il clipping rende gli aggiornamenti più robusti, evitando cambiamenti eccessivi in un singolo step. Allo stesso tempo, limita gli effetti di eventuali update sbagliati: la policy rimane vicina alla versione precedente, permettendo alle iterazioni successive di correggere gradualmente la direzione del gradiente senza destabilizzare l'apprendimento. È proprio questa robustezza uno dei punti di forza dell'algoritmo.

3.3 Pseudocodice

Prima di mostrare lo pseudocodice è utile presentare alcuni concetti utili:

- **Timestep** (t): è un singolo passo temporale nell'interazione agente–ambiente completa, ovvero l'episodio. Ad ogni timestep, l'agente osserva lo stato s_t, esegue un'azione a_t, riceve una ricompensa r_t e passa allo stato successivo s_{t+1}.

- **Transizione**: è la quadrupla (s_t, a_t, r_t, s_{t+1}), ovvero tutte le informazioni raccolte in una singola interazione agente–ambiente. Ad ogni timestep si genera una transizione.

- **Experience horizon** (T): rappresenta il numero di timestep utilizzati per raccogliere dati da ciascun attore per aggiornare la

policy. In pratica, indica quanto è lungo l'orizzonte temporale delle esperienze considerate per un singolo batch.

- **Batch**: è l'insieme di dati raccolti usati per aggiornare la rete in una iterazione. Se si hanno N attori in parallelo e ognuno di essi esegue T timesteps, i dati totali sono $N \cdot T$ transizioni. L'insieme delle esperienze raccolte costituisce quindi il batch.

- **Minibatch**: poiché il batch completo $N \cdot T$ può essere molto grande, per rendere più efficiente e stabile l'ottimizzazione viene diviso in sottoinsiemi più piccoli, chiamati minibatch, usati per calcolare gli aggiornamenti dei parametri.

- **Epoca** (K): rappresenta un passaggio completo di addestramento sul batch raccolto. Dopo aver ottenuto $N \cdot T$ transizioni, queste non vengono utilizzate una sola volta, ma riutilizzate in più epoche consecutive. In ciascuna epoca, il batch completo viene suddiviso in minibatch, e la funzione obiettivo viene ottimizzata sequenzialmente su ciascun minibatch. In questo modo, lo stesso batch di dati viene sfruttato più volte, in particolare K volte, migliorando la stabilità e l'efficienza dell'addestramento senza dover generare nuove interazioni con l'ambiente.

- **Iterazione**: corrisponde a un ciclo completo dell'algoritmo, che comprende la raccolta di dati dagli attori per T timesteps, la costruzione del batch $N \cdot T$, e l'ottimizzazione della policy per

K epoche.

È importante notare che l'ottimizzazione di cui si parla è quella che si ottiene aggiornando i parametri θ della policy con 2.2 utilizzando il gradiente della funzione obiettivo surrogata.

Algorithm 1 Proximal Policy Optimization (PPO)

for iteration = 1, 2, ... **do**
  for actor = 1, 2, ..., N **do**
    Eseguire la policy $\pi_{\theta_{\text{old}}}$ nell'ambiente per T timesteps;
    Calcolare le stime degli Advantage $\hat{A}_1, ..., \hat{A}_T$;
  end
  Ottimizzare la funzione surrogata $L^{CLIP}(\theta)$ per K epoche usando minibatch di dimensione $M \leq N \cdot T$;
  Aggiornare i parametri: $\theta_{\text{old}} \leftarrow \theta$;
end

[15]

Lo **pseudocodice** dell'algoritmo mostra che PPO utilizza lo stile actor-critic, con N attori che eseguono in parallelo la policy $\pi_{\theta_{\text{old}}}$ sull'ambiente ottenendo informazioni pari a un batch di dimensione $N \cdot T$. Successivamente viene effettuata l'ottimizzazione della funzione surrogata $L(\theta)$ con 2.2 per K epoche su minibatch di dimensione $M \leq N \cdot T$, permettendo di sfruttare più volte le stesse transizioni e migliorando la **sample-efficiency**[2] [15].

Si osservi che nelle prime epoche $\pi_\theta = \pi_{\theta_{\text{old}}}$, ovvero $r = 1$, gli aggiornamenti non vengono limitati e si impara subito dai dati raccolti. Nelle epoche successive, la policy può differire dalla vecchia, e

[2]indica quanto bene un algoritmo impara da pochi dati: più è sample-efficient, meno interazioni con l'ambiente servono per migliorare la policy.

il clipping può far annullare il gradiente per le azioni che portano la policy troppo lontano, rallentando l'aggiornamento su quei dati fino alla successiva iterazione, quando vengono raccolte nuove transizioni.

In pratica, l'approccio permette di apprendere in modo stabile e ripetuto dagli stessi dati, cosa che non sarebbe possibile con i normali metodi policy-gradient, dove ogni update può modificare troppo la policy.

3.3.1 Generalized Advantage Estimation (GAE)

Per la stima dell'advantage si utilizza la tecnica **Generalized Advantage Estimation (GAE)** [14], fondamentale perché senza di essa sarebbe impossibile calcolare l'advantage in modo accurato: il calcolo esatto richiederebbe infatti la conoscenza di tutte le ricompense future (3.2), non ottenibili durante l'addestramento.

La formula di GAE si basa sull'errore di differenza temporale δ_t, presentato in 2.5. L'**advantage stimato con GAE** combina queste δ_t in modo esponenzialmente scontato:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l} [14] \tag{3.7}$$

dove:

- $\lambda \in [0, 1]$ regola il peso delle differenze temporali future nella stima dell'advantage: valori vicini a 0 danno maggiore impor-

tanza all'errore immediato δ_t, producendo una stima più *locale*, mentre valori vicini a 1 tengono conto di molti errori futuri, producendo una stima più *globale* che integra informazioni su più passi temporali.

- $\gamma \in [0, 1]$ è il fattore di sconto, che ha la stessa funzione vista nell'errore di differenza temporale 2.5 e nel return scontato 1.2

- la combinazione $\gamma\lambda$ crea un **decadimento esponenziale**: le differenze temporali più lontane pesano meno, evitando di introdurre rumore eccessivo nella stima.

In pratica, GAE permette di ottenere un'ottima stima dell'advantage senza conoscere tutte le ricompense future, riducendo la varianza e migliorando la stabilità dell'algoritmo PPO.

Chapter 4

Caso di studio: atterraggio di un velivolo

In questo capitolo viene presentato il caso di studio scelto per valutare l'efficacia dell'algoritmo di reinforcement learning Proximal Policy Optimization (PPO), già analizzato dal punto di vista teorico nei capitoli precedenti. L'obiettivo è applicare l'algoritmo a un problema pratico di controllo autonomo: l'atterraggio di un velivolo in un ambiente simulato.

A tale scopo si è fatto uso di **MATLAB**[1], in particolare del *Reinforcement Learning Toolbox* [7], che mette a disposizione ambienti e strumenti per la modellazione, l'addestramento e la validazione di agenti intelligenti.

Nei paragrafi seguenti verranno quindi descritte nel dettaglio le

[1]ambiente di calcolo numerico e linguaggio di programmazione ad alto livello, utilizzato per analisi, simulazione e sviluppo di algoritmi scientifici e ingegneristici.

fasi del lavoro: dalla definizione dell'ambiente di simulazione e della funzione di ricompensa, alla progettazione e addestramento dell'agente PPO, fino alla valutazione delle prestazioni mediante simulazioni e all'analisi critica dei risultati ottenuti.

4.1 Definizione dell'ambiente e formulazione della ricompensa

Per l'implementazione si è fatto uso dell'ambiente simulato messo a disposizione dal Reinforcement Learning Toolbox: **Lander Vehicle**. Il Lander Vehicle è un ambiente di simulazione sviluppato in MATLAB con l'obiettivo di fornire un banco di prova per lo studio e l'addestramento di agenti intelligenti. Esso rappresenta un veicolo semplificato in due dimensioni, ispirato ai modelli classici di **lunar lander**[2], che deve compiere un atterraggio controllato in presenza della gravità, utilizzando esclusivamente due propulsori (thruster) posizionati simmetricamente rispetto al centro di massa.

L'ambiente Lander Vehicle è implementato come una classe completa che incorpora tutti gli elementi necessari alla definizione di un ambiente di reinforcement learning. inclusi lo stato, lo spazio delle azioni, il vettore delle osservazioni e la funzione di ricompensa, rendendolo immediatamente utilizzabile per l'addestramento di un agente.

[2]veicolo spaziale progettato per atterrare sulla superficie di un corpo celeste

4.1.1 Ambiente come classe

Il Lander Vehicle è implementato come una classe MATLAB e, in quanto tale, definisce attributi e metodi per consentire la simulazione.

Gli attributi si dividono in:

- **attributi costanti**: i parametri fisici del sistema, quali massa, accelerazione gravitazionale, dimensioni geometriche e limiti di spinta dei propulsori, pari a 8,5 Newton.
- **attributi variabili**: lo stato dinamico corrente del lander come posizione, velocità lineare e velocità angolare.

I metodi costituiscono l'interfaccia logica e funzionale dell'ambiente. Tra i principali si evidenziano:

- **`reset()`**: inizializza lo stato del veicolo in condizioni di partenza;
- **`dynamics(action)`**: ha lo scopo di descrivere la fisica del lander. Dato lo stato corrente e l'azione scelta, ovvero i comandi di spinta ai thruster, calcola le derivate delle variabili di stato secondo le equazioni differenziali del moto, che vengono poi integrate nel metodo `step` per aggiornare lo stato del sistema.
- **`step(action)`**: è il metodo principale dell'ambiente, invocato a ogni passo dell'episodio. Riceve l'azione scelta dall'agente e aggiorna lo stato interno calcolando le derivate tramite il metodo

`dynamics` e integrandole nel tempo. Successivamente restituisce la ricompensa, le nuove osservazioni e l'indicatore di fine episodio. In questo modo realizza la transizione stato–azione–nuovo stato tipica del paradigma di reinforcement learning.

Questa organizzazione rende il Lander Vehicle un ambiente modulare e facilmente richiamabile da qualsiasi algoritmo di reinforcement learning.

In MATLAB l'istanza dell'ambiente può essere creata semplicemente con:

```
env = LanderVehicle();
```

4.1.2 Spazio degli stati

La classe definisce anche lo **stato interno** del veicolo, descritto da un vettore di sei componenti:

$$\mathbf{s} = \begin{bmatrix} x & y & \theta & \dot{x} & \dot{y} & \dot{\theta} \end{bmatrix}^T$$

dove:

- x, y rappresentano la **posizione** del centro di massa;
- θ è l'**angolo di orientamento** rispetto alla verticale;
- $\dot{x}, \dot{y}$ sono le **velocità lineari**;
- $\dot{\theta}$ è la **velocità angolare**.

All'interno della classe è definito anche un **vincolo fondamentale** sull'ambiente: il movimento del lander è vincolato all'interno di un'area rettangolare, con l'asse orizzontale X compreso tra -100 e 100 metri e l'asse verticale Y compreso tra 0 e 120 metri.

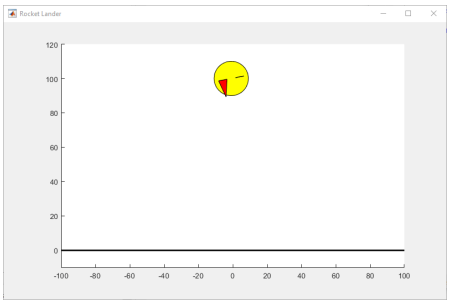


Figure 4.1: Rappresentazione grafica dell'ambiente nell'interfaccia di simulazione in MATLAB

4.1.3 Vettore di osservazioni

Come si è visto nei capitoli precedenti l'ambiente deve fornire a ogni passo un vettore di osservazioni. Questo vettore rappresenta la prima interfaccia tra agente e ambiente.

Il **vettore osservazione** fornito dalla classe Lander Vehicle è:

$$\mathbf{o} = \begin{bmatrix} x & y & \theta & \dot{x} & \dot{y} & \dot{\theta} & s \end{bmatrix}^T$$

dove le prime 6 componenti sono quelle viste prima mentre s è la lettura del sensore che indica la **condizione di atterraggio**:

- -1 per un hard landing,
- 1 per un soft landing,

- 0 se il veicolo è in volo.

Viene indicato un soft landing quando il veicolo tocca il suolo con velocità orizzontale e verticale inferiori a 0.5 m/s, altrimenti viene indicato un hard landing.

Tutte le componenti del vettore di osservazione sono **normalizzate** in un intervallo compreso tra -1 e 1, ovvero vengono scalate rispetto ai loro valori massimi e minimi attesi. Normalizzare rende le variabili confrontabili tra loro, mantiene numeri gestibili per la rete e aiuta l'agente ad imparare più efficacemente senza essere confuso da scale diverse delle variabili.

4.1.4 Spazio delle azioni

Lo spazio delle azioni costituisce la seconda interfaccia tra l'agente e l'ambiente, poiché rappresenta i comandi inviati dall'agente al veicolo sul come utilizzare i thruster. In generale lo spazio delle azioni potrebbe essere continuo, con valori reali compresi entro limiti prefissati, ma per semplificare il problema è stato adottato uno **spazio discreto**.

A ogni passo l'agente deve selezionare una delle **nove azioni predefinite**, che corrispondono a combinazioni di spinte dei due motori. Le intensità di spinta sono state normalizzate con i valori $L = 0.0$(spinta nulla), $M = 0.5$(metà della massima spinta) e $H = 1.0$(massima spinta); sarà poi la funzione `step` a convertirle nelle corrispondenti

Azione	$[T_1, T_2]$	**Descrizione**
1	[L, L]	Nessuna spinta
2	[L, M]	Solo thruster destro a livello medio
3	[L, H]	Solo thruster destro a livello alto
4	[M, L]	Solo thruster sinistro a livello medio
5	[M, M]	Entrambi i thruster a livello medio
6	[M, H]	Thruster sinistro a livello medio e destro a livello alto
7	[H, L]	Solo thruster sinistro a livello alto
8	[H, M]	Thruster sinistro a livello alto e destro a livello medio
9	[H, H]	Entrambi i thruster a livello alto

Table 4.1: Spazio delle azioni discrete del Lander Vehicle.

spinte fisiche.

Quindi il vettore azione fornito dall'agente è:

$$a = \begin{bmatrix} T_1 \\ \\ T_2 \end{bmatrix}$$

dove T_1 e T_2 rappresentano rispettivamente le spinte del motore sinistro e destro.

4.1.5 Definizione funzione ricompensa

L'**obiettivo** principale del *Lander Vehicle* è raggiungere la posizione di atterraggio corrispondente al punto $(0, 0)$ nel piano. In tale configurazione il veicolo deve inoltre mantenere un orientamento verticale nullo, ossia risultare parallelo all'asse y, condizione necessaria per garantire un atterraggio stabile.

Per guidare l'agente verso questo obiettivo, l'ambiente calcola una

funzione di ricompensa ad ogni passo temporale, fornendola direttamente all'agente. La ricompensa al passo t è definita come:

$$r_t = (s_t - s_{t-1}) - 0.1\theta_t^2 - 0.01(L_t^2 + R_t^2) + 500\, c$$

dove

$$s_t = 1 - \left(\sqrt{\hat{d}_t} + \frac{\sqrt{\hat{v}_t^2}}{2}\right), \quad \hat{d}_t = \frac{\sqrt{x_t^2 + y_t^2}}{d_{\max}}, \quad \hat{v}_t = \frac{\sqrt{\dot{x}_t^2 + \dot{y}_t^2}}{v_{\max}}$$

e

$$c = (y_t \leq 0) \wedge (\dot{y}_t \geq -0.5 \wedge |\dot{x}_t| \leq 0.5)$$

In questa formulazione:

- x_t, y_t, $\dot{x}_t$ e $\dot{y}_t$ sono rispettivamente le **posizioni** e le velocità del lander lungo gli assi x e y;
- $\hat{d}_t$ rappresenta la distanza del lander dalla posizione obiettivo $(0, 0)$, normalizzata rispetto alla massima distanza $d_{\max}$;
- $\hat{v}_t = \frac{\sqrt{\dot{x}_t^2 + \dot{y}_t^2}}{v_{\max}}$ rappresenta la velocità risultante del lander, normalizzata rispetto alla massima velocità $v_{\max}$;
- s_t è una misura di prestazione che utilizza distanza dall'obiettivo e velocità del lander per fornire un indice della qualità dello stato corrente;
- θ_t è l'orientamento del lander rispetto all'asse verticale;

- L_t e R_t sono i valori di azione scelti per i thruster sinistro e destro;

- c è una ricompensa sparsa assegnata in caso di *soft-landing*.

Per comprendere il funzionamento di questa funzione di ricompensa, è utile analizzare il contributo dei suoi singoli termini:

- $(s_t - s_{t-1})$: misura il miglioramento tra due istanti consecutivi. Se il lander si avvicina all'obiettivo e riduce la velocità, il termine è positivo e verrà fornito un premio; se invece si allontana o accelera, diventa negativo e verrà generata una penalità.

- $-0.1\theta_t^2$: penalizza deviazioni dall'assetto verticale. In questo modo l'agente è incentivato a mantenere il lander il più possibile orientato in modo stabile e parallelo all'asse y.

- $-0.01(L_t^2 + R_t^2)$: riduce la ricompensa proporzionalmente alla potenza utilizzata dai thruster. Ciò incoraggia manovre più leggere, evitando un uso eccessivo della propulsione.

- $+500c$: assegna un premio molto elevato in caso di *soft landing*. Questo termine spinge l'agente a completare l'episodio non solo centrando l'obiettivo, ma anche con una manovra di atterraggio sicura.

In sintesi, la funzione di ricompensa integra penalità e incentivi in modo complementare: da un lato orienta l'agente nel migliorare progressivamente la traiettoria e l'assetto del lander, dall'altro premia in

maniera decisiva il conseguimento di un atterraggio sicuro e controllato.

Inoltre, l'ambiente definisce anche la condizione di fine episodio: un episodio termina quando il lander tocca il suolo, collidendo con esso in maniera morbida o brusca, oppure quando il veicolo esce dai limiti dell'area di simulazione.

4.2 Creazione e addestramento dell'agente PPO

Per realizzare l'agente PPO e addestrarlo sul problema dell'atterraggio si è seguito l'approccio classico di cui si è parlato teoricamente nei capitoli precedenti, adattandolo allo stile e ai vincoli dell'ambiente `LanderVehicle`.

4.2.1 Architettura dell'agente

Essendo il PPO un algoritmo actor-critic, la prima cosa da fare è impostare le due reti neurali complementari: il **critic**, che prende in input il vettore delle osservazioni e restituisce in output il valore $V(s)$ che stima la ricompensa attesa dallo stato corrente, e l'**actor**, che riceve lo stesso vettore di osservazioni e produce in output una distribuzione di probabilità sulle 9 azioni discrete, ovvero un vettore di 9 elementi ognuno rappresentante la probabilità di eseguire una delle

diverse azioni. Tramite questo vettore viene poi campionata, ovvero scelta casualmente l'azione effettiva da eseguire.

Si sono utilizzate reti neurali di tipo **feedforward**[3] sia per l'actor che per il critic. Ciascuna rete è stata dotata di uno strato nascosto di grandi dimensioni, pari a 400 neuroni. Un'ampiezza del genere per lo strato nascosto permette all'agente di apprendere politiche più articolate e accurate, indispensabili per gestire la complessità dinamica del problema di atterraggio trattato.

4.2.2 Scelta degli iperparametri

Per ottenere un addestramento stabile ed efficiente, è importante scegliere valori opportuni per gli iperparametri.

Di seguito si riportano i principali iperparametri già introdotti nei capitoli precedenti, chiarendone il ruolo e la motivazione della scelta dei valori adottati:

- **Experience horizon**: numero di passi usati dall'agente prima di aggiornare la policy, impostato a 900 perché un valore elevato consente di raccogliere una quantità sufficiente di esperienza per rendere l'addestramento stabile e le stime dell'advantage più accurate.

- **Mini-batch e Numero di epoche**: essenziali per l'esecuzione

[3]Una rete feedforward è un tipo di rete neurale artificiale in cui l'informazione fluisce in modo unidirezionale dagli input verso gli output, senza retroazioni o cicli

dell'algoritmo 1. Sono stati scelti mini-batch di 300 campioni e 3 epoche per aggiornamento. Questa configurazione rappresenta un compromesso tra efficienza computazionale e aumento della stabilità.

- **Learning rate**: è il fattore di scala con cui vengono aggiornati i pesi delle reti neurali, introdotto in 2.2. Un valore eccessivo porterebbe a instabilità, mentre uno troppo basso rallenterebbe la convergenza. Sono stati adottati valori differenti per le due reti: $5 \cdot 10^{-4}$ per l'actor, in modo da aggiornare la policy in maniera graduale e stabile, e $9 \cdot 10^{-3}$ per il critic, che deve adattarsi più rapidamente per fornire stime di valore affidabili.

- **Fattore di clip**: già introdotto come fattore di clipping della funzione obiettivo del PPO in 3.6, limita la variazione della policy tra aggiornamenti successivi. È stato fissato a 0.04, valore che offre un buon equilibrio tra stabilità e capacità di adattamento della policy al problema specifico.

- **Fattore di sconto**: è il coefficiente γ già visto nell'equazione 2.5 e nel return scontato 1.2, che riduce progressivamente il peso delle ricompense future. Valori prossimi a 1 favoriscono strategie a lungo termine, mentre valori più bassi privilegiano guadagni immediati. Nel nostro caso è stato scelto $\gamma = 0.998$, molto vicino a 1, per dare maggiore importanza alle ricompense future,

favorendo l'apprendimento di strategie orientate al lungo termine.

Questi valori rappresentano un punto di partenza iniziale, ma possono essere adattati e ottimizzati in base ai risultati ottenuti sperimentalmente e alle capacità computazionali disponibili.

4.2.3 Istanziazione dell'agente

Una volta definite le reti neurali che costituiscono actor-critic e scelti gli iperparametri principali per l'addestramento, il passo successivo consiste nella **creazione effettiva dell'agente PPO**. In MATLAB questo avviene tramite il seguente frammento di codice:

```
rng(0,"twister");
agent = rlPPOAgent(obsInfo, actInfo, initOpts, agentOpts)
```

Il comando `rng(0,"twister")` serve a fissare il seme del generatore di numeri pseudocasuali. Infatti in MATLAB i numeri casuali sono prodotti da algoritmi deterministici, quindi a parità di seme, la sequenza generata sarà sempre la stessa. Questo è indispensabile per fare in modo che ad ogni episodio l'agente sia inizializzato con gli stessi valori per i parametri casuali, come i pesi delle reti neurali. Quindi fissando il seme si garantisce la **riproducibilità**: eseguendo lo stesso episodio più volte nelle stesse condizioni, si otterrano gli stessi risultati.

Inoltre questo serve per confrontare in maniera coerente i risultati dei diversi episodi.

La funzione `rlPPOAgent` permette invece di creare l'agente PPO vero e proprio, combinando tutte le componenti precedentemente definite [7]. In particolare:

- `obsInfo` e `actInfo` ottenibili dall'istanza dell'ambiente, specificano rispettivamente le caratteristiche dello spazio delle **osservazioni** e delle **azioni** dell'ambiente, garantendo che l'agente sia compatibile con il problema da risolvere;

- `initOpts` contiene le opzioni di inizializzazione delle reti neurali, come il numero di neuroni dello strato nascosto;

- `agentOpts` racchiude invece tutti gli **iperparametri** scelti in precedenza, che definiscono il comportamento dell'algoritmo di addestramento.

In questo modo, l'agente PPO viene istanziato e reso pronto per la fase successiva, ovvero l'addestramento all'interno dell'ambiente `LanderVehicle`

4.2.4 Procedura di addestramento

Per poter addestrare l'agente PPO è necessario definire alcune importanti opzioni di addestramento, utili a garantire un addestramento stabile, riproducibile ed efficiente. Si riportano di seguito le principali

opzioni utilizzate, con una descrizione della loro utilità e delle motivazioni dietro le scelte adottate.

- **MaxEpisodes=10000**: numero massimo di episodi di addestramento. Si è scelto un valore elevato per garantire che l'agente abbia sufficiente tempo per esplorare l'ambiente e migliorare progressivamente la propria policy, essendo il problema complesso.

- **MaxStepsPerEpisode=600**: numero massimo di singole interazioni agente-ambiente per episodio. Questo limite evita episodi troppo lunghi, assicurando un utilizzo efficiente delle risorse.

- **StopTrainingCriteria="AverageReward"**: viene impostato un ulteriore criterio per interrompere l'addestramento. In particolare, quest'utlimo si ferma anche quando la ricompensa media ottenuta dall'agente su un certo numero di episodi consecutivi raggiunge o supera la soglia definita da `StopTrainingValue` nel punto successivo. Questo permette di terminare il training non solo in base al numero di episodi, ma anche al raggiungimento di una performance desiderata.

- **StopTrainingValue=450**: definisce la soglia di ricompensa media che, una volta raggiunta o superata, determina l'interruzione automatica dell'addestramento. In questo modo si garantisce che l'agente apprenda una policy ottimale senza prolungare inutilmente il training.

- **ScoreAveragingWindowLength=100**: indica il numero di episodi considerati per calcolare la ricompensa media usata nel criterio di arresto alternativo. Si è optato per una finestra abbastanza grande, pari a 100 episodi, questo per evitare fluttuazioni casuali e rendere più affidabile la decisione di fermare l'addestramento.

Anche in questa fase è necessario fissare il seme dei generatori di numeri casuali con `rng(0,"twister")` per garantire la **riproducibilità** dei risultati. In questo caso fissare il seme serve a garantire che le scelte casuali effettuate durante l'addestramento, come il campionamento delle azioni esplorative seguano sempre lo stesso andamento.

L'addestramento dell'agente viene eseguito mediante la funzione **`train`**:

```
trainingStats = train(agent, env, trainOpts);
```

Durante l'esecuzione, l'agente interagisce con l'ambiente, raccoglie dati sugli stati, le azioni e le ricompense, e si aggiorna secondo l'algoritmo PPO.

Alla fine dell'addestramento si ottengono due risultati principali:

- **trainingStats**: un oggetto che contiene le informazioni sull'andamento dell'addestramento, come la ricompensa totale per episodio, il numero di episodi completati e eventuali criteri di arresto rag-

giunti. Questi dati sono utili per valutare l'efficacia della policy appresa e per confrontare diverse sessioni di addestramento.

- **agent aggiornato**: l'oggetto agente stesso viene modificato internamente durante l'addestramento. Vengono aggiornati i pesi delle reti neurali progressivamente e la policy dell'agente diventa più efficace nel massimizzare la ricompensa cumulativa.

4.3 Simulazione e analisi

Dopo l'addestramento, si ottiene un agente PPO pronto per essere testato nell'ambiente `LanderVehicle` attraverso simulazioni. Queste ultime servono a osservare il comportamento dell'agente in scenari controllati. In questa fase l'agente usa sempre la stessa policy per interagire con l'ambiente, senza aggiornarla, poiché quella ottimale è già stata appresa durante l'addestramento.

4.3.1 Procedura di simulazione

Prima di avviare la simulazione, è necessario definire alcune opzioni che ne regolano l'esecuzione. In particolare, è possibile impostare:

- **`MaxSteps`**: stesso parametro usato anche per l'addestramento dell'agente. Impostato a 600 come in precedenza per evitare episodi troppo lunghi.

- **`Numero simulazioni`**: il numero di episodi consecutivi da simulare, serve semplicemente ad eseguire più simulazioni senza dover ripetere il comando manualmente. Impostato a 5.

Inoltre richiamando il metodo **`plot`** dell'ambiente durante le simulaziomi verranno mostrate graficamente in tempo reale le manovre di atterraggio passo per passo, ovvero verrà mostrato l'evoluzione dello stato del veicolo ad ogni aggiornamento.

Una volta configurate le opzioni, si è pronti ad eseguire la simulazione vera e propria con il comando **`sim`**:

```
experience = sim(env, agent, simOptions);
```

Ad ogni passo della simulazione avviene quanto segue:

1. L'ambiente fornisce all'agente il vettore di osservazioni correnti.
2. L'agente calcola la distribuzione di probabilità sulle azioni tramite la rete actor e seleziona un'azione da eseguire casualmente.
3. L'ambiente aggiorna lo stato del lander utilizzando il metodo `step(action)` e calcola la ricompensa associata, che verrà poi fornita all'agente.
4. Il processo si ripete fino al raggiungimento del numero massimo di passi oppure alla condizione di fine episodio.

È importante notare che in questa fase la rete critic non ha nessun contributo, infatti essa viene utilizzata esclusivamente durante l'addestramento per far assumere alla rete complementare actor la forma ottimale.

Il risultato delle simulazioni è memorizzato nella variabile **`experience`**. Essa contiene tutte le informazioni dei singoli episodi, come stati, azioni e ricompense, permettendo di analizzare in maniera dettagliata le performance dell'agente.

4.3.2 Analisi dei risultati

Le cinque simulazioni effettuate mostrano un comportamento complessivamente **stabile** e **coerente** da parte dell'agente addestrato. In tutti i casi il lander riesce a ridurre progressivamente l'altitudine fino a raggiungere il suolo, mantenendo velocità verticali entro valori accettabili e senza mostrare manovre brusche.

Le simulaziomi partono da condizioni iniziali differenti e ciò influenza principalmente la dinamica orizzontale: la posizione e la velocità lungo l'asse x presentano differenze più marcate nelle fasi iniziali, ma tendono in ogni simulazione ad assestarsi attorno a valori prossimi allo zero. Analogamente, anche l'orientamento del veicolo viene stabilizzato nel tempo: l'angolo inizialmente oscilla, ma converge gradualmente verso lo zero, così come la velocità angolare. Tutto ciò evidenzia la capacità dell'agente di correggere efficacemente eventuali deviazioni

dall'assetto ottimale.

Un aspetto rilevante riguarda l'esito degli atterraggi: nella maggior parte delle prove l'agente riesce a completare la manovra con un soft landing, mentre in alcuni casi si verifica un hard landing. Ciò indica che la policy appresa è generalmente efficace, pur **non essendo perfetta**: la natura stocastica della policy e la variabilità delle condizioni iniziali possono infatti portare a manovre meno ottimali. È normale quindi che non tutti gli episodi abbiano l'esito sperato ed è proprio osservando la distribuzione tra successi e fallimenti che si può analizzare l'addestramento dell'aggiornamento per modificare i parametri di training per migliorare ulteriormente il comportamento appreso.

Nel complesso, i risultati evidenziano la capacità dell'agente PPO di generalizzare a diverse configurazioni di partenza, garantendo un **controllo stabile** e portando quasi sempre a un atterraggio sicuro del lander.

Chapter 5

Conclusioni

Il lavoro svolto ha permesso di approfondire il ruolo del reinforcement learning nel controllo autonomo di sistemi complessi, con particolare attenzione all'algoritmo Proximal Policy Optimization (PPO). Nonostante i metodi off-policy moderni possano essere più efficienti nell'uso dei dati, grazie alla capacità di riutilizzare più volte le esperienze passate tramite una particolare memoria, PPO rimane un punto di riferimento per la ricerca e le applicazioni pratiche. Essendo un algoritmo on-policy, infatti, deve raccogliere nuovi dati per ogni aggiornamento, ma compensa con la sua stabilità e la capacità di funzionare bene in una vasta gamma di ambienti, sia discreti sia continui.

L'analisi condotta e l'esempio presentato ha messo in evidenza tutto ciò. É stato mostrato come il PPO riesca a garantire un addestramento stabile e progressivo anche in un contesto complesso come l'atterraggio di un velivolo, caratterizzato da dinamiche continue e

non lineari, dove oscillazioni e divergenze avrebbero potuto compromettere la capacità di apprendimento dell'agente. Inoltre, l'uso della struttura actor-critic e il meccanismo di clip del PPO hanno permesso di ridurre la varianza negli aggiornamenti, rendendo l'apprendimento più efficiente rispetto ai metodi policy gradient puri.

Accanto a questi aspetti positivi, sono emerse alcune limitazioni. Infatti nonostante PPO sia progettato per essere più stabile di altri algoritmi, rimane sensibile agli iperparametri, specialmente il learning rate e il fattore di clip. Quindi l'utilizzo di nuove tecniche di ottimizzazione e adattamento dinamico degli iperparametri potrebbero rendere l'apprendimento più robusto e generalizzabile.

In conclusione, la tesi ha permesso di approfondire sia gli aspetti teorici sia quelli pratici del reinforcement learning, dimostrando l'efficacia di PPO come strumento di controllo autonomo. Inoltre i risultati ottenuti mostrano chiaramente come il reinforcement learning sia una tecnologia concreta e promettente, destinata a rivestire un ruolo sempre più centrale nello sviluppo di sistemi intelligenti e autonomi.

Ringraziamenti

Voglio ringraziare per prima cosa le persone con cui sono diventato Edgar, o Ed, ovvero Dario, Paki, Vobevto, Silvana, Angelica e Benedetta, con cui ho affrontato il liceo e ho ancora ora tra i ricordi più divertenti della mia vita. In particolare Dario e Paki, quelli seri, con cui è nata in quegli anni un'amiciza immensa, che mi dà ancora oggi sicurezza: nonostante non ci vediamo così spesso, so che anche tra 30 anni ci potremo contattare e, vederci sarà sempre come quando ci beccavamo nel mio parco.

Passiamo poi a quando sono diventato Tauto con Umberto, Stefano, Kekko, Marco, Dario, Paki, Sauro e Matteo. Vi voglio ringraziare perchè con voi ho iniziato a viaggiare, siamo scesi tutti i giorni per anni, abbiamo parlato delle nostre prime paure e dei nostri sogni, e sono stato offeso nei modi più strani possibili. In poche parole sono cresciuto con voi.

Non posso non fare un ringraziamento speciale a Dario e Stefano. Voi due siete diventati due pilastri fondamentali della mia vita: siete le prime persone a cui penso quando ho bisogno di un consiglio, spesso

accompagnati da una birra al Verdenova; non mi giudicate per qualsiasi scelta faccia e siete la valvola di sfogo per ogni problema. Spero che, rimanendo ancora così legati, tutti i sogni e gli obiettivi di cui abbiamo parlato in questi anni si realizzino per tutti e tre, non vedo l'ora di venire nel tuo Mcdonald Ste. Vi voglio bene ragazzi.

Negll'ulitmo anno ho trovato poi nuovi amici, che stranamante non mi hanno dato un nuovo soprannome. E' passato poco ma già sento una forte sintonia e l'ultimo anno di università è pesato poco anche grazie a voi. Con le ragazze, che anche ringrazio, è poi nato un **"madre che bello"** gruppo e sono molto contento.

È doveroso ringraziare le persone con cui ho vissuto questi tre anni: Giovanni, Matteo, Luca, Ivan, Giovanni, Alessando, Vincenzo, Marco e Giovanni. Con voi ho condiviso stress, gasamento prima degli esami, parigine bianche e tante soddisfazioni.

In particolare voglio ringraziare Luca, per aver risolto qualsiasi mio dubbio a qualsiasi ora, con degli audio che, devo ammettere, certe volte mi sembrava diventassero spiegazioni sull'origine dell'universo.

Però quando penso all'università, ormai la associo sempre a Matteo, detto Saccone. Abbiamo vissuto 3 anni in simbiosi e lo dovremo fare forse per altri 2 almeno. Per passare il tempo tra un esame e l'altro abbiamo parlato tantissimo degli argomenti più disparati: politica, trasporti, economia, senso dell'amore, storia, di tutto, anche se molte cose che mi hai detto so che erano supercazzole. Mi hai migliorato

come persona e sei il miglior collega-amico che mi potesse capitare.

Poi arriviamo a te, Kekka, ti ho incontrato nel periodo universitario e mi hai stravolto la vita: mi hai insegnato ad amare non solo nel senso di amore tra due fidanzati ma qualsiasi tipo di amore, mi hai fatto ritornare un bambino nonostante rinnegassi quella parte di me dandomi spensieratezza e facendomi apprezzare ciò che ho. Inoltre con te ho vissuto tra i ricordi più felici della mia vita che mi hanno dato la carica per affrontare le mie costanti paranoie, compresa l'università. So che a volte avresti voluto tirarmi un calcio nelle palle, ma mi hai sempre supportato, perciò questo risultato è un pò anche merito tuo. Anche se fai fatica ad ammetterlo, sei una persona fantastica, Ti amo.

Bibliografia

[1] K21 Academy. An introduction to reinforcement learning. https://k21academy.com/datascience-blog/machine-learning/reinforcement-learning/, 31/08/2025.

[2] Wikipedia contributors. https://en.wikipedia.org/wiki/Proximal_policy_optimization, 31/08/2025.

[3] Wei Di, Jianing Wei, and Anurag Bhardwaj. *Deep Learning Essentials*, pages 1–284. Packt Publishing, 2018.

[4] Emilio Greco and Antonio Francesco Gentile. Modellazione di oggetti cognitivi per la gestione energetica efficiente e la qualità indoor in edifici intelligenti. Technical Report RT-ICAR-CS-20-01, CNR - Istituto di Calcolo e Reti ad Alte Prestazioni (ICAR), March 2020.

[5] Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *Proceedings of the 19th In-*

ternational Conference on Machine Learning (ICML), volume 2, pages 267–274. Morgan Kaufmann, 2002.

[6] MathWorks. `https://it.mathworks.com/campaigns/offers/guide-to-understanding-reinforcement-learning-html`, 31/08/2025.

[7] MathWorks. `https://it.mathworks.com/products/reinforcement-learning.html`, 31/08/2025.

[8] MathWorks. `https://it.mathworks.com/help/reinforcement-learning/ug/proximal-policy-optimization-agents.html`, 31/08/2025.

[9] MathWorks. `https://it.mathworks.com/help/reinforcement-learning/ug/train-discrete-ppo-agent-to-land-vehicle.html`, 31/08/2025.

[10] OpenAI. `https://spinningup.openai.com/en/latest/algorithms/ppo.html`, 31/08/2025.

[11] OpenAI, 2017. `https://openai.com/blog/openai-baselines-ppo/`, 31/08/2025.

[12] OpenAI, 2019. https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html#a-taxonomy-of-rl-algorithms, 31/08/2025.

[13] Stuart Russell and Peter Norvig. *Intelligenza artificiale: un approccio moderno*, pages 1–672. Pearson, 2021.

[14] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2018. https://arxiv.org/abs/1506.02438, 31/08/2025.

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[16] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction to*, pages 1–552. Bradford Books, 2018.

[17] Csaba Szepesvári. *Algorithms for Reinforcement Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2010.

[18] Ryan K. Tan, Yang Liu, and Lei Xie. Reinforcement learning for systemspharmacology-oriented and personalized drug design. *Expert Opinion on Drug Discovery*, 17:849–863, 2022.

[19] Lilian Weng, 2018. https://lilianweng.github.io/posts/2018-04-08-policy-gradient/, Blog: Lil'Log, 31/08/2025.